National University of Computer and Emerging Sciences
Islamabad Campus

Assignment 03
Identified Vulnerabilities and Fixes

CY-4001

Secure Software Design

Submitted by: M. Huzaifa Fayyaz, Abdul Nafay Azam
Zeeshan Qaiser, Abdul Samad

Roll number: 22i-1692, 22i-1765, 22i-1773, 22i-1702

Date: 26th Oct 2025

Table of Contents

- ## Identified Vulnerabilities and Fixes

Objective:

This report details the identification and rectification of common security vulnerabilities within the provided web application. The focus is on applying secure coding practices to address these issues and demonstrating improvements through a combination of detailed explanations and code examples.

1.Password Security (BCrypt Integration)

Vulnerability:

The original application stored user passwords in plaintext and compared them directly during authentication. This practice is highly insecure as it makes user credentials vulnerable to compromise if the database is breached. Attackers gaining access to the database would immediately have cleartext passwords, leading to potential account takeovers.

Solution:

To address this, BCrypt hashing was integrated. BCrypt is a strong, adaptive password hashing function that incorporates a salt and allows for a configurable work factor, making brute-force attacks computationally intensive.

1. A **BCryptPasswordEncode**r bean was configured in **SecurityConfig.java**.
2. The **DataSeeder.java** was updated to use this PasswordEncoder to hash the passwords of initial (seeded) users before saving them to the database.
3. The authentication flow in **AuthController.java** was modified to use **passwordEncoder.matches()** to securely compare the provided password with the stored hash, without ever handling the plaintext password directly.

SecurityConfig.java

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

DataSeeder.java

```
AppUser u1 = users.save(AppUser.builder().username("alice").password(passwordEncoder.encode("alice12345"
```

AuthController.java

```
if (!passwordEncoder.matches(req.getPassword(), user.getPassword())) {
    return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(Collections.singletonMap("error", "invalid credentia
}
```

2. Access Control (SecurityFilterChain)

Vulnerability:

The initial SecurityFilterChain in SecurityConfig.java was overly permissive, potentially allowing unauthenticated access to sensitive **/api/**** endpoints due to broad permitAll() configurations or insufficient authorization rules. This could expose internal application logic or sensitive data to unauthorized users.

Solution:

The SecurityFilterChain was strengthened to enforce a "deny by default" policy and implement role-based access control.

1. Specific endpoints were explicitly permitted **(e.g., /api/auth/** for authentication, /h2-console/** for development, and POST /api/users for public user registration).**
2. All other requests (anyRequest()) were configured to **authenticated()**, requiring a valid JWT for access.
3. Role-based access control was enforced for administrative endpoints, requiring the **ADMIN** role for **/api/admin/**.**
4. Custom **AuthenticationEntryPoin**t and **AccessDeniedHandler** were introduced to provide consistent JSON error responses fo**r 401 Unauthorized** and **403 Forbidden** errors, respectively, improving API usability and security.

3. Resource Ownership Enforcement

Vulnerability:

Without proper ownership checks, a malicious user could potentially access or manipulate data belonging to other users by simply guessing or incrementing resource IDs **(e.g., /api/accounts/{otherUserId}/balance)**. This is a critical authorization bypass vulnerability.

Solution:

In key controller methods, mechanisms were implemented to ensure that users can only access or modify resources they own.

1. A helper method **getAuthenticatedUserId()** was created to reliably extract the userId from the authenticated principal.
2. Before processing requests for specific resources (e.g., retrieving an account balance or initiating a transfer), the extracted **authenticated** userId is compared with the ownerUserId of the requested resource.
3. If the authenticated user does not own the resource, a **403 Forbidden** response is returned, preventing unauthorized access.

AccountController.java

```java
@GetMapping("/{id}/balance")
public ResponseEntity<?> balance(@PathVariable Long id, Authentication auth) {
    Long authenticatedUserId = getAuthenticatedUserId(auth);
    return accountRepository.findById(id).map(account -> {
        if (!account.getOwnerUserId().equals(authenticatedUserId)) {
            return ResponseEntity.status(HttpStatus.FORBIDDEN).body(Collections.singletonMap("error", "Access denied"));
        }
        // ... more code ...
```

UserController.java

```java
@GetMapping("/{id}")
public ResponseEntity<?> get(@PathVariable Long id, Authentication auth) {
    Long authenticatedUserId = getAuthenticatedUserId(auth);
    if (!id.equals(authenticatedUserId)) {
        return ResponseEntity.status(HttpStatus.FORBIDDEN).body(Collections.singletonMap("error", "Access denied. You can or
    }
    // ... more code ...
```

4. Data Exposure Control (DTO Implementation)

Vulnerability:

The API endpoints were directly returning entity objects (e.g., AppUser, Account), which often contain sensitive fields such as password, role, or **isAdmin.** Exposing these fields in API responses, even if encrypted in the database, poses a significant risk as it reveals internal system structure and sensitive user information.

Solution:

Data Transfer Objects (DTOs) were implemented to explicitly control which data is exposed via the API.

1. **UserDTO.java** was created to expose only safe user information (id, username, email).
2. **AccountDTO.java** was created to expose safe account information (id, iban, balance).
3. Controller methods **(UserController.java, AccountController.java)** were updated to map entity objects to their respective DTOs before returning them in API responses. This prevents sensitive fields from ever leaving the server.

```java
// src/main/java/edu/nu/owaspapivulnlab/web/dto/UserDTO.java
@Data
@NoArgsConstructor
@AllArgsConstructor
public class UserDTO {
    private Long id;
    private String username;
    private String email;
}
```

```java
// src/main/java/edu/nu/owaspapivulnlab/web/dto/AccountDTO.java
@Data
@NoArgsConstructor
@AllArgsConstructor
public class AccountDTO {
    private Long id;
    private String iban;
    private Double balance;
}
```

UserController.java

```java
        return userRepository.findById(id)
                .map(user -> ResponseEntity.ok(new UserDTO(user.getId(), user.getUsername(), user.getEmail())))
                .orElseGet(() -> ResponseEntity.status(HttpStatus.NOT_FOUND).body(Collections.singletonMap("error", "Resource not found"
    }
```

5. Rate Limiting

Vulnerability:

Critical and sensitive endpoints **(e.g., /api/auth/login, /api/accounts/{id}/transfer)** were vulnerable to brute-force attacks, denial-of-service, or abuse due to an absence of rate limiting. This could allow attackers to guess credentials, rapidly exhaust resources, or spam functionality.

Solution:

A custom RateLimitingFilter.java was implemented based on a token bucket algorithm to limit the number of requests to sensitive endpoints within a given time frame.

1. The **RateLimitingFilter** is a **OncePerRequestFilter** that intercepts requests.
2. It uses a **ConcurrentHashMap** to maintain a token bucket state for each client (identified by IP address for login, or by authenticated user for transfer).
3. The filter refills tokens periodically and consumes a token for each request. If no tokens are available, a **429 Too Many Requests** response is returned.
4. The filter was integrated into **SecurityConfig.java** with a high precedence, ensuring it runs before authentication and authorization checks, **returning 429 early** if the limit is exceeded. Rate limit parameters (capacity, refill rate) are configurable **via application.properties.**

RateLimitingFilter.java

```
public RateLimitingFilter(@Value("${app.rate.limit.capacity}") long capacity,
                          @Value("${app.rate.limit.refill-duration-seconds}") long refillDurationSeconds,
                          @Value("${app.rate.limit.refill-tokens}") long refillTokens) {
    this.capacity = capacity;
    this.refillDurationMillis = refillDurationSeconds * 1000;
```

RateLimitingFilter.java

```
    if (isRateLimitedEndpoint(requestURI, requestMethod)) {
        String key = getKey(request, requestURI);
        TokenBucketState bucket = buckets.computeIfAbsent(key, k -> new TokenBucketState(capacity, System.currentTimeMillis()));

        synchronized (bucket) {
            long currentTime = System.currentTimeMillis();
            long timeElapsed = currentTime - bucket.lastRefillTime;
            if (timeElapsed > refillDurationMillis) {
                long refills = timeElapsed / refillDurationMillis;
                bucket.tokens.set(Math.min(capacity, bucket.tokens.get() + refills * refillTokens));
                bucket.lastRefillTime = currentTime;
            }
```

```
            if (bucket.tokens.get() > 0) {
                bucket.tokens.decrementAndGet();
                filterChain.doFilter(request, response);
            } else {
                response.setStatus(HttpStatus.TOO_MANY_REQUESTS.value());
                response.setContentType(MediaType.TEXT_PLAIN_VALUE);
                response.getWriter().write("Too many requests. Please try again later.");
                return; // Crucial to terminate the filter chain here
            }
        }
    } else {
        filterChain.doFilter(request, response);
    }
```

SecurityConfig.java

```
            http.addFilterBefore(new RateLimitingFilter(rateLimitCapacity, rateLimitRefillDurationSeconds, rateLimitRefillTokens), org.s
```

6. Mass Assignment Prevention

Vulnerability:

Without explicit control over incoming data, an API could be vulnerable to mass assignment. This occurs when an attacker includes extra, unauthorized fields in a request (e.g., setting "isAdmin": true during user registration), and the application blindly binds these fields to the backend object, leading to privilege escalation or unintended data modification.

Solution:

Explicit request DTOs were used for all incoming data, and server-side validation was applied.

1. **UserCreationRequestDTO.java** was created specifically for user registration, containing only the username, password, and email fields. It explicitly excludes sensitive fields like role or isAdmin.

2. **TransferRequestDTO.java** was created for transfer requests, including only the amount field.
3. **Controller methods** were updated to accept these specific DTOs using **@RequestBody @Valid,** ensuring only the intended fields are processed and validated. Any extra fields in the request body are ignored.

```java
// src/main/java/edu/nu/owaspapivulnlab/web/dto/UserCreationRequestDTO.java
@Data
@NoArgsConstructor
@AllArgsConstructor
public class UserCreationRequestDTO {
    @NotBlank(message = "Username is required")
    @Size(min = 3, max = 50, message = "Username must be between 3 and 50 characters")
    private String username;

    @NotBlank(message = "Password is required")
    @Size(min = 8, max = 100, message = "Password must be between 8 and 100 characters")
    private String password;

    @Email(message = "Invalid email format")
    @NotBlank(message = "Email is required")
    private String email;
}
```

UserController.java

```java
    @PostMapping
    public ResponseEntity<?> create(@Valid @RequestBody UserCreationRequestDTO request) {
        if (userRepository.findByUsername(request.getUsername()).isPresent()) {
            return ResponseEntity.status(HttpStatus.CONFLICT).body(Collections.singletonMap("error", "Username already exists"));
        }
        // ... more code
```

7. JWT Hardening

Vulnerability:

The security of JWT-based authentication was vulnerable due to potential use of weak secret keys, excessively long token lifetimes (TTL), and a lack of validation for standard claims like issuer and audience. This could lead to tokens being easily forged, replayed, or used for extended periods if compromised.

Solution:

The JwtService.java was significantly hardened to improve JWT security:

1. **Strong Secret Key:** The JWT secret key is now loaded from an environment variable (app.jwt.secret) using **@Value**, ensuring it is not hardcoded and can be securely managed. This secret is used to derive a SecretKey for **HMAC-SHA signing.**
2. **Short Token Lifetime (TTL):** A short token lifetime **(app.jwt.ttl-seconds=900, i.e., 15 minutes)** is configured in **application.properties** and enforced during token issuance, minimizing the window of opportunity for replayed or compromised tokens.

3. **Issuer and Audience Claims:** issuer and audience claims are now included during token creation and are strictly validated during token parsing. This helps prevent tokens issued by other systems or intended for other services from being accepted.
4. **Strict Validation:** The parseToken() method strictly validates the token's signature, expiry, issuer, and audience using **Jwts.parserBuilder().** Any deviation results in a JwtException, leading to a 401 Unauthorized response.

application.properties

```
app.jwt.secret=your_strong_jwt_secret_from_env_variable
app.jwt.ttl-seconds=900
```

JwtService.java

```
public JwtService(@Value("${app.jwt.secret}") String secret) {
    if (secret == null || secret.isEmpty() || secret.length() < 32) { // Minimal length check for a strong secret
        throw new IllegalArgumentException("JWT secret must not be empty and should be at least 32 characters long.");
    }
    this.secretKey = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
}
```

JwtService.java

```
public Claims parseToken(String token) throws JwtException {
    return Jwts.parserBuilder()
            .setSigningKey(secretKey)
            .requireIssuer(issuer)
            .requireAudience(audience)
            .build()
            .parseClaimsJws(token)
            .getBody();
}
```

8. Error Handling & Logging

Vulnerability:

In a production environment, detailed error messages, stack traces, and internal class names returned to clients can expose sensitive information about the application's internal workings, aiding attackers in reconnaissance and exploiting vulnerabilities. Lack of secure server-side logging can hinder incident response.

Solution:

A comprehensive error handling and logging strategy was implemented to provide generic error messages to clients while logging full details server-side.

1. **Reduced Error Detail:** application.properties was configured to prevent exposing internal messages and stack traces to clients (**server.error.include-message=never, server.error.include-stacktrace=never)**.

2. **Custom Exception Mapping (GlobalErrorHandler.java):** An **@ControllerAdvice class**, GlobalErrorHandler, was created to centralize exception handling. It maps various exceptions (e.g., RuntimeException, **MethodArgumentNotValidException**, **DataAccessException**) to generic, client-friendly JSON error responses.
3. **Consistent JSON Responses:** Spring Security's AuthenticationEntryPoint **(jsonAuthenticationEntryPoint())** and AccessDeniedHandler **(jsonAccessDeniedHandler())** were configured in SecurityConfig.java to ensure 401 Unauthorized and 403 Forbidden responses are consistently returned as JSON. The JwtFilter also ensures its 401 responses are JSON.
4. **Secure Server-Side Logging:** The GlobalErrorHandler logs full exception details (stack traces, specific messages) internally at an appropriate level **(WARN/ERROR)** using a Logger, which is invaluable for debugging and monitoring without exposing these details externally.

application.properties

```
server.error.include-message=never
server.error.include-stacktrace=never
```

GlobalErrorHandler.java

```
@ExceptionHandler(RuntimeException.class)
public ResponseEntity<?> handleRuntimeException(RuntimeException e, WebRequest request) {
    logger.error("RuntimeException: {}", e.getMessage(), e);
    return createErrorResponse(HttpStatus.INTERNAL_SERVER_ERROR, "An unexpected error occurred.");
}
```

SecurityConfig.java

```
http.exceptionHandling(eh -> eh
        .authenticationEntryPoint(jsonAuthenticationEntryPoint())
        .accessDeniedHandler(jsonAccessDeniedHandler())
);
```

9. Input Validation

Vulnerability:

Without robust input validation, APIs are susceptible to various attacks, including SQL injection, cross-site scripting (XSS), and logic flaws. Accepting invalid, negative, or excessively large numeric values (e.g., in transfer amounts) or malformed strings can lead to unexpected behavior, data corruption, or security breaches.

Solution:

Comprehensive server-side input validation was implemented across all critical user inputs.

1. **Jakarta Validation Annotations:** jakarta.validation annotations **(e.g., @NotBlank, @Size, @Email, @NotNull, @DecimalMin(value = "0.01"))** were extensively used on fields within

request DTOs **(UserCreationRequestDTO.java, TransferRequestDTO.java)** and the LoginReq inner class in **AuthController.java**.

2. **@Valid Annotation:** The **@Valid annotation** was applied to **@RequestBody** parameters in controller methods **(UserController.java, AccountController.java, AuthController.java)**. This triggers the validation process automatically.
3. **Error Handling for Validation:** The GlobalErrorHandler.java includes an **@ExceptionHandler for MethodArgumentNotValidException.class**, which captures validation failures and returns appropriate 400 Bad Request responses with user-friendly error messages derived from the validation annotations. This ensures that invalid inputs are rejected early in the request processing pipeline.

```java
@Data
@NoArgsConstructor
@AllArgsConstructor
public class UserCreationRequestDTO {
    @NotBlank(message = "Username is required")
    @Size(min = 3, max = 50, message = "Username must be between 3 and 50 characters")
    private String username;

    @NotBlank(message = "Password is required")
    @Size(min = 8, max = 100, message = "Password must be between 8 and 100 characters")
    private String password;

    @Email(message = "Invalid email format")
    @NotBlank(message = "Email is required")
    private String email;
}
```

```java
@Data
@NoArgsConstructor
@AllArgsConstructor
public class TransferRequestDTO {
    @NotNull(message = "Transfer amount is required")
    @DecimalMin(value = "0.01", message = "Transfer amount must be positive")
    private Double amount;
}
```

AuthController.java

```java
@NotBlank(message = "Username is required") @Size(min = 3, max = 50, message = "Username must be between 3 and 50 characters")
private String username;
```

AuthController.java

```java
public ResponseEntity<?> login(@RequestBody @Valid LoginReq req) {
```

Test Cases

After implementing all the fixes, the following Maven commands were used in the project's root directory to build the application and run the integration tests to verify the secure behaviors:

1. **To Clean and Build the Project:** This command compiles the code, packages it, and runs any unit tests, ensuring all dependencies are resolved and the project is in a clean state.

   mvn clean install
   mvn spring-boot:run

2. **To Run All Integration Tests (or Specific Test Class):** This command executes the integration tests, including AdditionalSecurityExpectationsTests.java, which verify the implemented security fixes.

   mvn test

These commands were run repeatedly during the development and debugging process until all tests passed successfully, confirming the effective resolution of the identified vulnerabilities.